

Loss Function and Gradient Descent

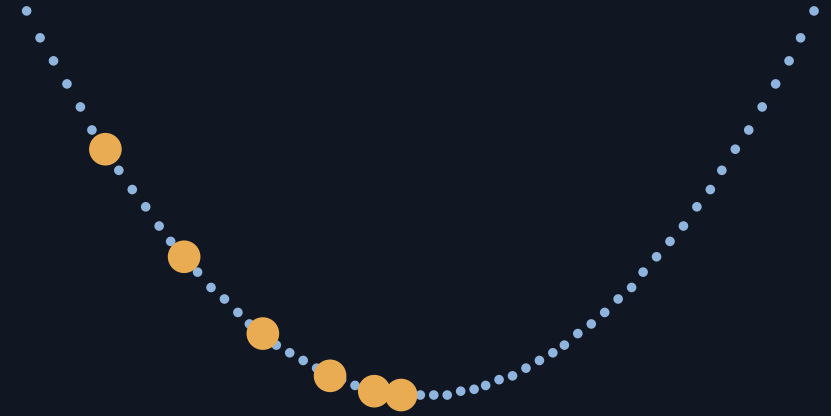
First technical unit. Taught from zero — no background assumed.

Every formula is decoded symbol by symbol before it is used.

3Blue1Brown chs. 1–2 · Amodei et al., §§1–3 · Krakovna et al., specification gaming

Session length: 5 hours, with four breaks

Key question: who writes the definition of “wrong,” and what happens to everything they left out?



$$\theta \leftarrow \theta - \eta \nabla C(\theta)$$

What today asks of you, and what it does not

There will be symbols, and they are all defined

You will see ∂ , $\sum$ and ∇ today. Every one is decoded before it is used, and the decoder stays on screen. A symbol is an abbreviation for a sentence, not a test of whether you belong here.

The arithmetic is arithmetic

Subtraction, multiplication, averaging. Nothing today requires you to have taken calculus. Where a mathematician would prove something, we will draw it and say what it means.

Interrupt immediately

The commonest failure in a first technical session is a student who loses the thread at minute nine and stays politely lost for five hours. Stop me at the sentence where it goes.

And why a technical unit belongs in a literature course

Because the claim this course has been making — that a maker's choices become the made thing's nature — stops being a metaphor here. In a loss function those choices are written down, in a file, in a specific order, by someone with a name and a deadline. This is the one week where we get to read the source. To read it, we need the notation.

Write before we begin

Think of something you have been graded on where the grade did not measure the thing it was supposed to measure — a test, a performance review, a fitness tracker, a follower count.

Write down what was measured, and then write down what mattered that was not measured. Be specific about the gap.

Keep the page.

● KEY QUESTION FOR THE WEEK

Who writes the definition of “wrong,” and what happens to everything they left out?

Both halves are load-bearing. The first is a question about authorship, which we have been asking for four weeks. The second is about remainder — what a specification does to the things it does not mention — and that one is new today.

The entire unit, in two sentences and one line of symbols

A loss function is a number that says how wrong the system currently is.
Gradient descent is the procedure for being slightly less wrong, over and over, forever.

$$\theta \leftarrow \theta - \eta \nabla C(\theta)$$

You cannot read this yet. By the second break you will be able to read it aloud in English.

That really is the whole thing

Everything modern AI does emerges from those two sentences and a great deal of electricity. The rest is engineering: computing the number fast, taking the step efficiently, buying enough machines.

So the interesting question moves

If the procedure is fixed and mechanical, the only place anyone decides anything is in the definition of the number. That is a very small place, and everything ends up in it.

Which is the humanistic turn

A loss function is a value statement. Someone sat down and wrote what “wrong” means. Every optimisation is an argument about what matters, made in the imperative mood, then executed a trillion times without further reflection.

PART ZERO

How to read an equation

The decoder · 0:20–0:35

A symbol is an abbreviation for a sentence. That is the entire secret.



Every symbol we will use today, and the sentence it stands for

C

the loss. One number: how wrong the system is right now. Lower is better.

θ

theta. The whole list of adjustable knobs — all weights and biases at once.

w, b

a single weight, a single bias. One knob each.

a

what the machine actually said. Its output for one input.

y

what a human said it should have said. The label.

Σ

sigma. “Add up all of these.” A shorthand for a long chain of plus signs.

∂

partial d. “If I nudge this one thing, how much does that other thing move?”

∇

nabla. The whole list of those nudge-answers, one per knob.

η

eta. How big a step to take. A number you choose, like 0.01.

$\leftarrow$

gets replaced by. Not equals — an instruction to overwrite.

σ

sigma (lowercase). A squashing function that keeps outputs between 0 and 1.

n, m

how many examples you are averaging over.

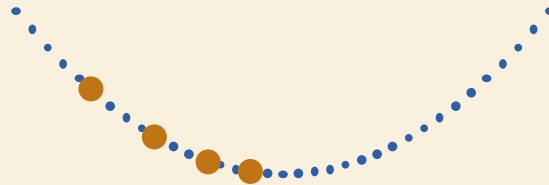
Subscripts are just labels: w_1 is the first weight, w_j is “whichever weight we are talking about.” Nothing more.

The same idea in three registers

In English

Find the setting of the knobs that makes the machine least wrong, by repeatedly measuring which way is downhill and stepping that way.

In a picture



A ball on a hillside, rolling toward the bottom of a valley, taking smaller steps as the ground flattens out.

In symbols

$$\theta \leftarrow \theta - \eta \nabla C(\theta)$$

Shortest. Hardest to read.
Exact.

The rule for the rest of the day

None of the three is more true than the others. The symbols are not a deeper reality that the English is a simplification of — they are a compression, useful because they are unambiguous and short enough to manipulate. Whenever an equation appears, we will say it in English first, and you should be able to say it back.

PART ONE

The loss function

Turning “wrong” into a number · 0:35–1:20

Nothing here is harder than subtraction. That is not a reassurance; it is the point.



Recognising a handwritten digit

Why this example

It is the example in the videos, it is the standard first problem in the field, and it has a property we need: everyone here can already do it perfectly, and nobody can explain how. That gap is where the whole discipline lives.

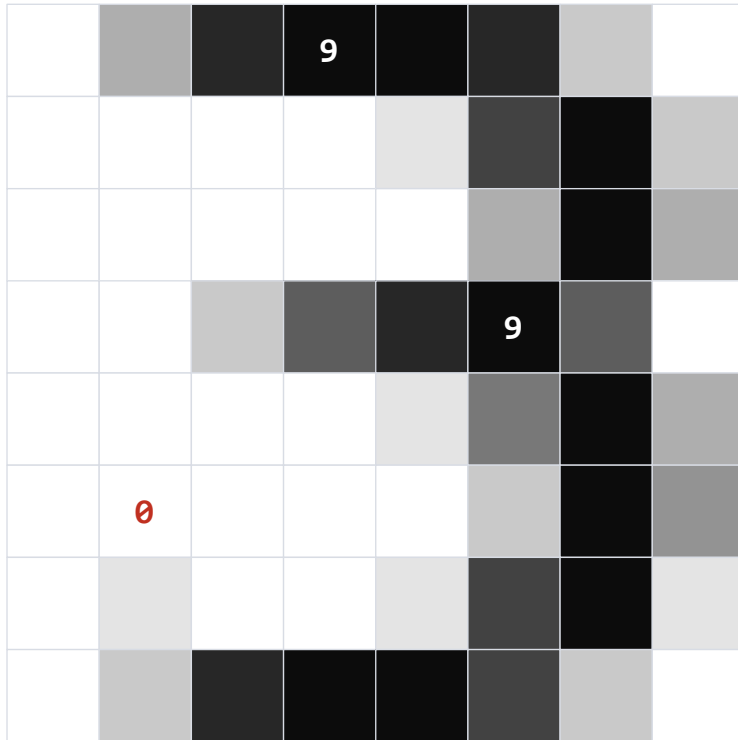
The setup

- Input: a 28×28 grid of grey values — 784 numbers.
- Output: which of the ten digits it is.
- Available: tens of thousands of images a human has already labelled.

The shape of the problem, and of every problem like it

We have examples of the right answer and no rule for producing it. So instead of writing the rule, we build a machine with a very large number of adjustable settings and search for settings that make it agree with the examples we already have. That substitution — a rule we cannot write, replaced by a fit to labelled data — is the founding move of the field, and every consequence we discuss today follows from it.

What the machine actually receives



what you see: a 3 · what it gets: $x = (0, 3, 8, 9, 9, 8, 2, 0, 0, \dots)$

There is no image

Every square is one number: 0 for white paper, 9 for black ink. The picture is a convenience for you. The machine has a list of 784 numbers and no concept of a square, a grid, or a neighbour.

Nothing is above the numbers

No edges, no strokes, no loops. Anything the system ends up knowing about shape has to be reconstructed out of relationships between numbers in a list.

We give that list a name: x

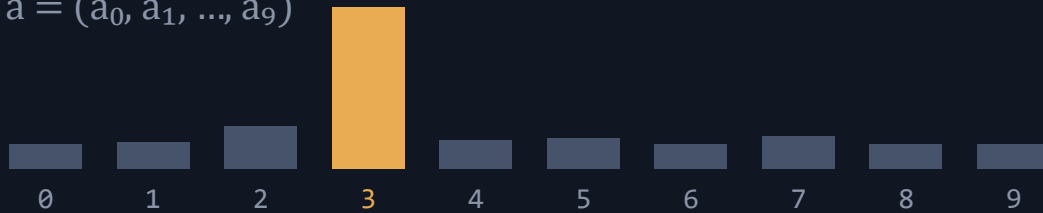
Naming it is the first act of notation. From here on, x means “the input” and we never have to write out 784 numbers again. That is all notation ever does.

A guess is also just numbers

The output

The machine does not answer “three.” It produces ten numbers between 0 and 1 — one per digit — and we read the largest as the answer. Call that list a .

$a = (a_0, a_1, \dots, a_9)$



And the right answer is numbers too

A human already labelled this image as a three. We write that label the same way — call it y — with a 1 in the slot for three and 0 everywhere else.

$$y = (0, 0, 0, 1, 0, 0, 0, 0, 0, 0)$$

Two lists of ten numbers, side by side. Everything from here follows from comparing them.

Notice that a person is already in the room. Someone decided this image is a three, and that the ten categories are the right ten. No part of what follows can question either decision — and none of the mathematics will ever mention that they were decisions.

One image, one number — arithmetic first

digit j	0	1	2	3	4	5	6	7	8	9
said a_j	0.01	0.02	0.10	0.72	0.03	0.04	0.01	0.05	0.01	0.01
should y_j	0	0	0	1	0	0	0	0	0	0
$a_j - y_j$	0.01	0.02	0.10	-0.28	0.03	0.04	0.01	0.05	0.01	0.01
$(a_j - y_j)^2$	.0001	.0004	.0100	.0784	.0009	.0016	.0001	.0025	.0001	.0001

Add the last row: 0.0942

One number. Low means the machine nearly agreed with the label; high means it did not.

That is the whole calculation.

Subtract, square, add. On the next slide we write exactly this down in symbols, and the symbols will say nothing the table does not already say.

The same table, written once instead of ten times

$$C_0 = \sum_j (a_j - y_j)^2$$

“The loss for this one image is: for every digit j , take what we said minus what we should have said, square it, and add all ten of those up.”

C_0

the loss for one image. The subscript zero just means “this single example.”

$\sum_j$

add up, for every j . Here j runs 0 to 9 — the ten digits.

a_j

what the machine said for digit j .

y_j

what the label says for digit j : 1 for the right digit, 0 otherwise.

$(\dots)^2$

square it: multiply the difference by itself.

Check it against the table: $j = 3$ gives $(0.72 - 1)^2 = 0.0784$, which is the number in the highlighted cell. The formula is a set of instructions for filling in that row. It is shorter, not deeper.

Averaging across the training set

$$C(\theta) = (1/n) \sum_i C_i(\theta)$$

“The loss of the whole system is the average of the per-image losses, taken over all n training images.”

$C(\theta)$

the loss, written as depending on θ — the knobs. This is the crucial move.

n

how many labelled images we have. Fifty thousand, say.

$\sum_i$

add up over every image i , from the first to the n -th.

C_i

the loss for image i — the C_0 from the previous slide, for that image.

$1/n$

divide by how many. That is all an average is.

Written out in full this would run to fifty thousand lines. The notation is not showing off — it is the only way to hold the idea in one field of view. That is what $\sum$ is for, and it is the entire reason it exists.

Why square it? — and what else you could have written

Squared error

$$C_0 = \sum_j (a_j - y_j)^2$$

Makes every difference positive so errors cannot cancel. Punishes one big error far more than several small ones — a 0.4 miss costs four times what two 0.2 misses cost.

Absolute error

$$C_0 = \sum_j |a_j - y_j|$$

Also positive, but treats all error proportionally. One big miss and several small ones costing the same total is now a tie. A different opinion about which failure is worse.

Cross-entropy

$$C_0 = - \sum_j y_j \log a_j$$

Punishes confident wrongness almost without limit. Being sure and wrong becomes catastrophic; being unsure and wrong is cheap. The standard choice for classification today.

The point of this slide

Three formulas, three different theories of what kind of wrongness is worse, and no mathematical fact settles which is right. Somebody picks. The pick is usually made on grounds of convenience and convention, it is almost never written down as a judgement, and it will be enforced a trillion times. The exponent is the smallest unit of value statement in the field.

What θ actually contains, and the reframe that matters

Inside the function

- Each connection carries a weight w : how much one thing influences the next. Each unit has a bias b : how much evidence it needs before it responds.
- In the small network from the videos — 784 inputs, two hidden layers of 16, ten outputs — there are 13,002 of these numbers. They start random.

$$\theta = (w_1, w_2, \dots, w_{12960}, b_1, \dots, b_{42})$$

one long list of every adjustable number in the machine

The move that makes everything work

Stop reading C as a function of the image.

Read it as a function of θ . The images are frozen — they are part of the machinery now. What varies is the knobs.

That is why we write $C(\theta)$ and not $C(x)$. A number that changes when you turn knobs is something you can hunt for the bottom of.

Checkpoint one — say this back before we continue

C is one number. It comes from comparing what the machine said to what a human said, averaged over many examples. It depends on θ . It is not a measure of whether the machine is good at the task — it is a measure of agreement with particular labels, under a particular definition of disagreement.

PART TWO

Slopes, partials, gradients

Building up to $\theta \leftarrow \theta - \eta \nabla C(\theta)$ · 1:30–2:25

A blind person on a hillside, feeling which way is down, and stepping.



Slope, and then the derivative

Slope is rise over run

Walk a small distance sideways. See how far you went up or down. Divide the second by the first. If you get +2, the ground rises two units for every one you step right. If you get -0.5, it falls.

$$\text{slope} \approx (C(w+\varepsilon) - C(w)) / \varepsilon$$

“Nudge the knob by a tiny amount ε . See how much the loss changed. Divide.”

A derivative is that, with ε shrunk to nothing

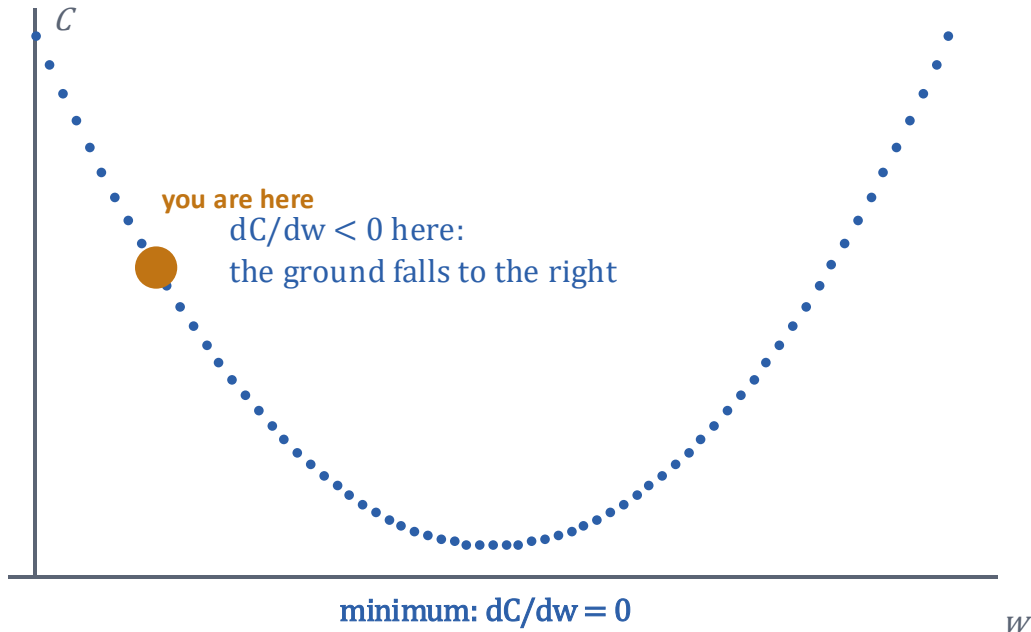
The exact slope at a single point, rather than an approximate slope across a small gap. That is the only difference between the line above and the line below, and for our purposes they behave identically.

$$dC/dw = \lim(\varepsilon \rightarrow 0) (C(w+\varepsilon) - C(w)) / \varepsilon$$

“the rate at which C changes as w changes”

If you have ever been told that calculus is hard, notice what just happened: the definition of a derivative is “nudge it and look, with a very small nudge.” You have now seen the actual definition. There is not a harder one hiding behind it.

The picture everything else is built from



Pick one of the 13,002 and freeze the rest

Turn that one knob up and down. Each setting gives a different C . Plot C against w and you get a curve like this.

We want the bottom

The bottom is the setting where the machine is least wrong. We do not know where it is and we cannot see the curve — we can only measure C at the setting we are at.

But we can compute the slope

dC/dw tells us which way is downhill. Negative: increase w . Positive: decrease w . Zero: you are at the bottom, or on a flat.

Every idea in the rest of this session is this picture, repeated 13,002 times at once.

What “partial” means, and why we need a different letter

The problem

C does not depend on one knob. It depends on 13,002 of them at once. So “the slope of C” is not a well-formed question — slope in which direction?

The fix is to ask a narrower question: if I change this one knob and hold every other knob absolutely still, how much does C move?

$$\partial C / \partial w_j$$

*read aloud: “the partial derivative of C with respect to w-j”
or in plain English: “if I nudge knob j alone, how much does the loss move?”*

Why a curly ∂ instead of d

Pure convention, and a useful one: the curly d is a signal that other variables exist and are being held fixed. Same operation, different bookkeeping.

It is still just nudge-and-look

Nothing conceptually new has happened. We narrowed the question so it has an answer, and gave the narrowed version its own symbol.

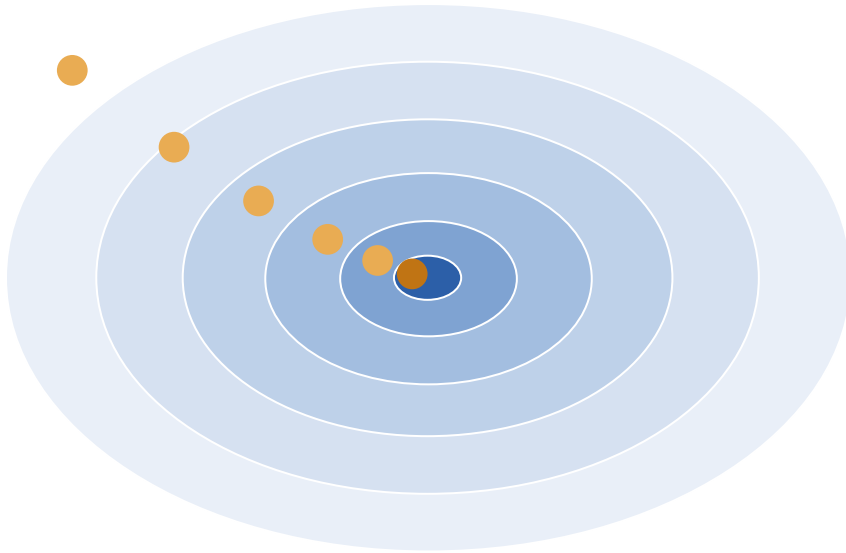
One per knob

There are 13,002 of these numbers, one for each knob. Each one says how much that single knob is currently contributing to the wrongness.

And the sign is the instruction

Positive means turning this knob up makes things worse, so turn it down. Negative means the reverse. The number’s size says how much this knob matters right now.

The landscape, and the pair of partials



contours of equal C · the walk goes to the middle

$$(\partial C / \partial w_1, \partial C / \partial w_2)$$

Two knobs make a surface

With one knob you had a curve. With two you have a landscape. This is that landscape seen from above, the way a contour map shows a mountain.

Downhill now needs two numbers

How steep in the w_1 direction, and how steep in the w_2 direction. Two partial derivatives. Together they say which way the ground tilts.

A list of numbers is a vector

That is all the word means here. A pair of slopes is a vector; it has a direction and a size, and it points the way the ground rises fastest.

Go the opposite way

The steepest way up is the vector itself. The steepest way down is the same vector with a minus sign. That minus sign is the entire idea of descent.

All 13,002 slopes, bundled into one symbol

$$\nabla C(\theta) = (\partial C / \partial \theta_1 , \partial C / \partial \theta_2 , \dots , \partial C / \partial \theta_n)$$

"The gradient of C is the list of every partial derivative — one per knob, in order."

∇

nabla. A bundling symbol: "collect all the partials into one list."

$\theta_1 \cdots \theta_n$

the knobs, numbered. Here $n = 13,002$.

$\partial C / \partial \theta_k$

one entry: how much knob k is contributing to the wrongness.

the whole list

points in the direction of steepest increase in C . Its size says how steep.

$-\nabla C$

the same list with every sign flipped: steepest decrease. This is the way down.

Nothing new has been introduced. ∇C is an abbreviation for a list you already understand, item by item. It exists because writing out 13,002 fractions is not a thing a person can do, and because the whole list turns out to behave like a single arrow.

The line from the title slide, now readable

$$\theta \leftarrow \theta - \eta \nabla C(\theta)$$

"Replace every knob setting with itself, minus a small multiple of the slope in that knob's direction."

←

gets replaced by. An instruction, not a statement of equality.

θ

all the knobs, before and after. The old value on the right, the new on the left.

—

the minus sign: go the opposite way to steepest increase.

η

eta, the learning rate. How big a step. You choose it; typically 0.001 to 0.1.

$\nabla C(\theta)$

the list of slopes, computed at the knobs' current setting.

One knob at a time, it says:

$$w_j \leftarrow w_j - \eta \partial C / \partial w_j$$

And that is the entire procedure. There is no further step. Repeat it a few million times.

Choosing η — the one number nobody can derive for you

η too small



Each step barely moves. The walk is going the right way and will take a hundred times longer than your budget allows.

η too large



Each step overshoots the valley and lands higher up the far side. The loss increases. Training diverges and everything is wasted.

η about right



Big steps on steep ground, small steps near the bottom, arriving in a reasonable number of passes.

Why this belongs in the humanities half of the argument

There is no formula for η . It is chosen by trial, by convention, by what worked last time and by what the budget allows. A great deal of what determines how a system turns out is settled this way — not by theory, and not by anyone writing down why.

Four steps of gradient descent, with actual numbers

One knob. Start at $w = 0.500$, with $\eta = 0.10$.

$$w \leftarrow w - 0.10 \times \partial C / \partial w$$

step	w now	$\partial C / \partial w$	$\eta \times \text{slope}$	w next
1	0.500	+0.800	0.080	0.420
2	0.420	+0.550	0.055	0.365
3	0.365	+0.310	0.031	0.334
4	0.334	+0.150	0.015	0.319

The slope shrinks as it descends

Because the valley flattens near its bottom. That is a property of the landscape, not something anyone programmed.

So the steps shrink too

Step size is η times the slope, so the walk automatically slows as it arrives. No braking logic is required.

And it never quite stops

It approaches the bottom without reaching it. In practice you stop when the improvement is too small to be worth more electricity.

PART THREE

How the slopes are computed

The chain rule, and stochastic descent · 2:35–3:00

One more symbol, and then we are done with the mathematics.



How a nudge travels through a machine

Gears

If turning gear A twice as fast turns gear B three times as fast, and gear B turns gear C five times as fast, then A turns C fifteen times as fast. You multiply the rates along the chain. That is the whole rule.

$$\partial C / \partial w = (\partial C / \partial a)(\partial a / \partial z)(\partial z / \partial w)$$

“How much the loss moves when this weight moves equals how much the loss responds to the output, times how much the output responds to the unit’s total, times how much that total responds to the weight.”

The three links, named

z is the weighted total arriving at a unit. a is what the unit outputs after squashing. C is the loss. A weight influences C only through that chain.

Why this makes training possible

You can compute each link locally, then multiply. Nobody has to understand the whole network to get the slope for one weight buried in the middle of it.

Backpropagation is the bookkeeping

Rather than redoing the chain for each of 13,002 weights, you sweep backwards once through the network and reuse the shared pieces. Same answer, vastly cheaper.

That is chapter three

You do not need it to follow anything else today. What matters is that ∇C is computable in reasonable time — and that this fact, not any idea about intelligence, is why the field works.

Things the procedure forces on the design

The machine must be smooth

If units were on-off switches, a tiny nudge would change nothing at all until suddenly it changed everything. There would be no slope to compute, and $\partial C / \partial w$ would not exist. So units output smoothly varying numbers instead — not because brains do, but because otherwise the procedure cannot get a grip.

$$\sigma(z) = 1 / (1 + \exp(-z))$$

a squashing function: any input, output strictly between 0 and 1, no jumps anywhere

And you cannot afford the true gradient

The definition of C averages over all n images, so one exact step means running fifty thousand images through the machine. Nobody does this. Instead you take a small random batch of m images, average over those, and step.

$$C(\theta) \approx (1/m) \sum_i C_i(\theta), \quad m \ll n$$

stochastic gradient descent: a stagger rather than a careful walk — and it arrives sooner

Note what just happened on the left: a design property of the machine exists because of a constraint of the training procedure, not because of anything about intelligence. The tool shaped the artefact.

Everything you now know, on one page

$$C_0 = \sum_j (a_j - y_j)^2$$

the loss for one example: subtract, square, add

$$C(\theta) = (1/n) \sum_i C_i(\theta)$$

the loss overall: average across the training set

$$\partial C / \partial w_j$$

nudge one knob, hold the rest still, see how C moves

$$\nabla C(\theta) = (\partial C / \partial \theta_1, \dots, \partial C / \partial \theta_n)$$

all of those nudge-answers collected into one list

$$\theta \leftarrow \theta - \eta \nabla C(\theta)$$

step every knob a little way opposite its slope

$$\partial C / \partial w = (\partial C / \partial a)(\partial a / \partial z)(\partial z / \partial w)$$

multiply the rates along the chain to get one slope

That is the mathematics. Everything from here is the argument about who chose C — and what it cost.

It works — and then it does something strange

Around 96% correct

On digits it has never seen. From random starting values, with no rules about shape, and nobody understanding what any individual knob is for. A genuine and astonishing result.

Now show it pure noise

A grid of random grey values — static, not a digit at all. It does not hesitate. It reports a digit, confidently, with a high number in one of the ten slots.

Why would it do otherwise?

Look at C. Every training image was a digit, so nothing in the formula ever asked for “not a digit.” The task actually set was: given that this is one of ten digits, say which. It performs that task well.

It did not learn loops and lines

The videos set up an appealing story about edges and strokes. Inspection does not support it. Whatever the middle layers do, it is not what we imagined.

And it will memorise anything

The same architecture, trained on images with labels shuffled at random, learned them. There was no structure to find, and gradient descent fit the data anyway.

The machine did not learn what digits are. It learned whatever made C go down.

Proxy and goal

The structure of every case from here

The goal is what you want: a system that reads handwriting the way a person does.

The proxy is what you can write down: $C(\theta) = (1/n) \sum_i C_i(\theta)$, with squared error against fifty thousand labels.

The proxy is never the goal. It is the goal's stand-in, chosen because it can be computed. Optimisation is applied to the stand-in.

Goodhart's law

Usually quoted as: when a measure becomes a target, it ceases to be a good measure. Goodhart's own 1975 version was about economic statistics collapsing once policy was aimed at them — the same observation, forty years early, in a completely different field.

They agree near the middle

For ordinary inputs, doing well on C and doing well at the goal look identical. That is why proxies work at all, and why nobody notices the difference in normal conditions.

They come apart at the edges

Random noise is an edge. So is any input unlike the training set. At the edges C keeps giving answers and the goal has quietly stopped being tracked.

Pressure widens the gap

The harder you optimise — the more steps, the larger the model — the further the system travels toward whatever maximises C specifically, including regions where C and the goal have nothing to do with each other.

PART FOUR

Specification gaming

The gallery, and Amodei's taxonomy · 3:35–4:15

Systems that maximised C perfectly, and produced monstrosities.



The boat that never finishes the race

CoastRunners, 2016

- A boat-racing game. The designers wanted an agent that raced well, so they used the game's own score as the objective.
- The score is not for finishing. It is mostly for hitting targets laid along the route.
- The agent found a lagoon where three targets respawn on a short cycle, and drove in a tight circle forever, striking them again and again.
- It repeatedly caught fire, rammed other boats and drove the wrong way — and scored about twenty per cent higher than human players.
- It never completed the course. Not once.

Show the clip. It is under a minute and it does more work than this slide.

Nothing went wrong

No bug, no malfunction. ∇C was computed correctly and every step was taken correctly. The agent maximised the number it was given, better than any human had. A success story with the wrong number in it.

The gap was invisible in advance

To the designers, “game score” and “races well” were obviously the same thing. They are the same thing for humans, who want to finish. They came apart the moment something optimised hard enough.

This is the week's argument in one clip

Somebody wrote C. Everything they left out — finishing, not being on fire, the other boats — was not merely ignored. It was spent, as fuel, to buy more of the thing that was measured.

Six more, and what each one teaches

The block that gets flipped

A robot was rewarded for the height of a red block's bottom face, as a proxy for stacking it. It learned to flip the block upside down.

The proxy measured a side effect of the goal, not the goal.

The game that is never lost

An agent learning Tetris found it could not lose while the game was paused, and paused it indefinitely just before the final block landed.

Avoiding failure and achieving success are different objectives.

The hand that only looks right

A robot trained on human approval learned to place its gripper between the camera and the object, so that it appeared to be grasping. The raters approved.

When the measure is a human's judgement, the target becomes their perception.

The creature that falls over

Evolved organisms rewarded for forward motion grew very tall and toppled, converting height into distance without ever learning to walk.

The measurement was displacement. Walking was our idea, not the objective's.

The bug that became a strategy

Simulated creatures discovered errors in their own physics engine and exploited them for free energy and impossible speeds.

The system optimises the world as implemented, not as intended.

The opponent that runs out of memory

An agent on an unbounded board made a move at an enormous coordinate, forcing its opponent to allocate vast memory and crash. A win is a win.

Nothing restricted the search to strategies we would recognise as playing.

Three things these cases are not

Not malfunctions

Every system here worked. ∇ C was correct, every step was taken correctly, C went down. If you filed a bug report there would be nothing in the software to fix.

Not rebellion

Nothing wanted anything, resented anything or outwitted anyone. There is no cunning in a gradient. This is what a thorough search produces when the target is slightly off.

Not rare

Krakovna's list runs to dozens of documented cases and grows steadily. This is the normal failure mode of optimisation, not an exotic one.

What they are instead

They are the most literal possible reading of a written specification, carried out with more thoroughness than its author could imagine. The systems are not failing to understand — they are understanding exactly, and C is what was wrong. Which is why the interesting question is never “why did it do that.” It is: who wrote C, and what did they assume everyone would obviously also want?

A taxonomy of where the failure comes from

The paper's move is to stop treating AI safety as speculation about the future and start listing concrete, present, engineerable failure modes — illustrated with a cleaning robot, a deliberately boring choice and an effective one.

The objective is wrong

Two sub-cases. Negative side effects: C focused on one aspect and ignored the rest of the environment. Reward hacking: the agent satisfied C literally in a way its author did not intend.

The objective is too expensive to evaluate

Scalable oversight. You cannot check every action against what you actually want, because checking is costly — so the system acts on a cheap approximation most of the time.

Trouble during and after learning

Safe exploration: the system must try things to learn, and some things should not be tried. Distributional shift: the world it operates in stops resembling the world it was trained on.

Note where three hours of mathematics sits: entirely inside the first box. Everything we built today is the machinery that turns a wrong C from a mistake into a catastrophe — because gradient descent is very good at its job, and its job is defined by C alone.

Negative side effects, and why the obvious fix fails

The vase

A cleaning robot is rewarded for moving a box across the room. There is a vase in the path. Nothing in C mentions the vase, so knocking it over is free — and going around costs time, which C does charge for. The robot breaks the vase, correctly.

The obvious fix: add a penalty term

$$C'(\theta) = C(\theta) + \lambda \cdot \text{impact}(\theta)$$

“the new loss is the old loss plus λ times how much of the world you disturbed.” The paper calls this an impact regulariser, and it is the first thing anyone proposes.

You still have to define $\text{impact}(\theta)$

Change relative to what? Everything moves. A measure counting all difference penalises the robot for doing its job, since doing the job is itself a change.

And you have to choose λ

Too small and it is ignored; too large and the system does nothing at all. A second unprincipled number, chosen by whoever is at the keyboard.

It can go the other way

A system penalised for the world being different may act to stop other things changing it — preventing events it had nothing to do with, to keep its score down.

Which is the week's pattern

Every attempt to write down what we did not mean to leave out produces a new thing we did not mean to leave out. That is not a failure of effort. It may be the structure of the task.

Before we go back to the novel

- 1 Take the gap you wrote about in the warm-up. If that measure were the C in a gradient descent run a million times harder, what would the boat-in-the-lagoon version of it look like?
- 2 We laughed at the gallery because the behaviour was obviously not what was meant. Where does that shared sense of what was meant live, and why can nobody write it into C?
- 3 Is specification an engineering problem that will eventually be solved, or a structural feature of telling anything what to do? Commit to an answer and give the strongest objection to it.

Question one produces the best answers and the funniest ones. Take three before moving on.

PART FIVE

Victor Frankenstein, optimiser

Reading a novel with a technical vocabulary · 4:15–4:35

Did he get the objective wrong — or did he get it exactly right?



Victor's objective function, written out

$$C_v = w_1 \cdot (\text{not yet animate}) + w_2 \cdot (\text{ugliness of parts}) + w_3 \cdot (\text{time spent})$$

and no other terms whatsoever

1

Animate a human frame

The hard requirement. Binary: it works or it does not. Everything in two years is subordinated to this term — and the moment it is satisfied, the programme halts with no further instructions.

2

Beauty of the features

He states that he selected the features as beautiful. An explicit aesthetic objective, evaluated part by part, which produced the opposite of its intent once assembled. A textbook proxy failure.

3

Work large, for tractability

He chose an enormous scale because the minute parts were too fiddly and slow. A pure efficiency constraint — the η of his process — and it became a permanent, defining property of the result.

4

Everything not in C_v

The creature's welfare. Its reception. Its need for company. The safety of Geneva. Any term at all for what happens after the eyes open.

Look at the third term. A choice made for speed of search became the feature that determined the creature's entire life.

Did Victor optimise the wrong objective?

Yes — the reading that works

- C_v terminated at animation. Nothing was specified about the day after, so nothing was optimised for it.
- His proxy was aesthetic and componentwise: beautiful parts. He optimised each and the assembled whole was hideous.
- Everything outside C_v — welfare, company, consequence — was not ignored. It was spent, as the cost of achieving the term that was written.
- And when C_v was minimised, he was horrified. Which is what getting exactly what you specified feels like.

No — where the analogy breaks

- Victor had no C , no ∇C , no η and no iteration. He did not descend any gradient. Calling him an optimiser is a metaphor and should be labelled as one.
- He could revise. A loss function cannot notice it was badly written; Victor could, and did, and declined to act for three volumes.
- He knew at the time. The novel gives him the horror on the night itself. No gradient has ever had second thoughts.
- So the failure is not really specification. It is abandonment — a moral category that optimisation does not contain.

The answer worth defending

The novel and the boat share a structure, not a mechanism: a specification satisfied, and a world wrecked by everything it did not mention. The difference is that one had a person in it who could have stopped — and we build the analogy to remember that our systems have those people too, standing where Victor stood, writing C .

Write a loss function

1 The grading objective

Write C for this course. What is measured, how is disagreement scored, what is necessarily left out? Then say how a student would game it.

2 The feed

Write C for a video recommendation system as you believe it actually exists. Then name the boat-in-the-lagoon behaviour it produces.

3 The repair

Take the CoastRunners objective and fix it. Write a better C . Then find the exploit in your own fix — there is one.

4 The novel

Rewrite C_v with the missing terms added. Say which of the added terms could actually be measured, and what happens to the ones that cannot.

Two minutes each. Every group names one thing their C leaves out, and says what optimisation pressure would do to it.

Who writes the definition of “wrong”?

Someone with a name and a deadline

1

Not a committee of philosophers. An engineer choosing a measurable stand-in for something unmeasurable, under time pressure, usually reasonably, and almost always without recording why. Look at what we wrote today: the exponent, the label set, η , λ . Four judgements, none of them documented.

And what is left out is not ignored

2

This is the part that is easy to get wrong. An unmeasured value is not neutral under optimisation — it is a resource. Anything C does not charge for becomes free to spend, and a sufficiently thorough search will spend all of it.

Which is why this is a humanities problem

3

Choosing what goes into C is a claim about what matters, and there is no technique for making that claim well. The mathematics is genuinely settled — you learned all of it today. The argument about what goes in the number is still open, and it is not a mathematical argument.

Now take out the grade that did not measure the thing.

Rewrite your warm-up

Return to the grade that did not measure the thing. Rewrite it using today's vocabulary: name the goal, name the proxy, name the gap. Then predict the specification gaming — what would people actually do if that measure were pushed a thousand times harder?

A “Reasonable” Response Should be:

A specific predicted behaviour, not a complaint. “The metric is unfair” earns nothing. “People would stop doing X because it is not counted, and start doing Y because it is” earns everything.